

Ezhuthu: A Stateful Tamil99 Keyboard Visualiser

Modelling Tamil99 Keyboard layout as a finite-state transducer

Lakshmanshankar

`lakshmanshankar.c@gmail.com`

<https://github.com/lakshmanshankar/ezhuthu>

October 6, 2026

Abstract

Tamil99 is a stateful keyboard layout: the output produced by a key depends on the key sequence preceding it. For example, the vowel key for ஐ produces the independent vowel ஐ when no consonant is pending, but the corresponding vowel sign ஐ when a consonant precedes it. Consequently, a static keyboard layout cannot fully describe its behavior.

This report models Tamil99 as a small Mealy machine whose state consists solely of the pending consonant, or the absence of one. A single pure transition function, `step`, implements the machine's transition and output function: given an initial state and a key input, it produces both the emitted text and the resulting state. The same transition function governs both the preview handler and the input mechanism, ensuring that the visualized and actual behavior remain consistent.

Contents

1	Introduction	3
2	Background	3
3	Formal Model	3
3.1	Alphabets	3
3.2	States	4
3.3	Transition and output	4
4	Layout Data	5
5	The step Function	5
6	Visualiser Design	7
6.1	Preview table	7
6.2	Architecture	8
6.3	Rendering Tamil text	8
7	Testing	8
8	Limitations and Future Work	8
9	Conclusion	8

1 Introduction

Most layout viewers (including those built on XKB data) map each physical key to one character. That is enough for Latin layouts but not for Tamil99, where one key can produce different output depending on context.

The goal is to show, at every moment, *what each key would do if pressed next*. After the user types ஃ , the vowel keys are relabelled ஃஶ , ஃகி , ஃகீ , ... instead of ஃஆ , ஃஇ , ஃஈ , ...

Components

- A formal model of Tamil99 as a finite-state transducer with 24 states.
- A pure, UI-independent **step** function.
- A preview table generated from **step**, so display and typing always agree.

2 Background

Tamil is an abugida. A consonant carries an inherent vowel அ ; other vowels are written with dependent signs, and the pulli (U+0BCD) suppresses the inherent vowel. A syllable is therefore often several code points:

$$\text{ஃஶ} = \text{U+0B95} + \text{U+0BCA}, \quad \text{ஃ} = \text{U+0B95} + \text{U+0BCD}.$$

Text should therefore be handled by grapheme cluster, not by code point. Note, however, that the default Unicode cluster rules do not treat Tamil conjuncts such as ஃஶ and ஃஶ் as one unit (see the Testing section), so the engine tracks these itself.

Tamil99 gives each vowel *one* key, which must act as an independent vowel at the start of a syllable and as a vowel sign after a consonant. This depends on the previous key, so the layout cannot be a static map.

3 Formal Model

We model Tamil99 as a Mealy machine $M = (Q, \Sigma, \Gamma^*, \delta, \lambda, q_0)$.

3.1 Alphabets

- $\mathcal{C}$: the 23 consonants that take vowel signs: 18 on the base layer plus 5 Grantha letters on the Shift layer (ஐ , ஶ , ஸ , ஹ , ஶ்). Note that ஶ் consists of three code points but behaves as a single consonant.
- $\mathcal{V}$: the 12 independent vowels.
- S : the special character ஃஶ் , which is emitted directly and has no corresponding vowel-sign form.
- Σ : input keys, identified by `(KeyboardEvent.code, shift)`:

$$\Sigma = K_{\mathcal{C}} \cup K_{\mathcal{V}} \cup K_S \cup \{k_{\text{pulli}}\}.$$

- Γ : Unicode code points; outputs are strings in Γ^* .

The layout data supplies $\kappa : K_{\mathcal{C}} \rightarrow \mathcal{C}$, $\nu : K_{\mathcal{V}} \rightarrow \mathcal{V}$, and $\rho : K_S \rightarrow S$, together with the vowel-sign table $\sigma : \mathcal{V} \rightarrow \Gamma^*$.

3.2 States

The only information that affects future output is the *pending bare consonant*: the consonant just typed that has not yet received a vowel sign or pulli.

$$Q = \{\varepsilon\} \cup \mathcal{C}, \quad q_0 = \varepsilon, \quad |Q| = 24.$$

ε means “nothing pending” (start of text, after a vowel, a vowel sign, a pulli, ᲡᲣᲚ , or a space). The state does not need the whole prefix: ᲡᲣᲚ and ᲡᲣ both lead to ε .

3.3 Transition and output

The pure function **step** takes the current state q and an input key k , and returns the emitted token together with the next pending state:

$$\text{step}(q, k) = \{\text{ins} = \lambda(q, k), \text{pending} = \delta(q, k)\}.$$

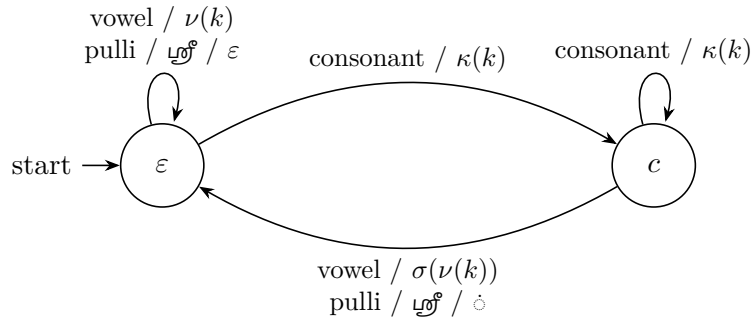
The function performs only the state transition and token emission. It does not combine a vowel sign with a pending consonant. The preview function uses the emitted token and the pending state to determine the resulting vowel-sign combination when displaying the key’s effect. The same **step** function is used by both the preview and input paths, keeping their state transitions consistent.

$$\lambda(q, k) = \begin{cases} \kappa(k) & k \in K_{\mathcal{C}} \\ \nu(k) & k \in K_{\mathcal{V}}, q = \varepsilon \\ \sigma(\nu(k)) & k \in K_{\mathcal{V}}, q \in \mathcal{C} \\ \text{Ს} & k = k_{\text{pulli}}, q \in \mathcal{C} \\ \varepsilon & k = k_{\text{pulli}}, q = \varepsilon \end{cases} \quad (1)$$

and the state-transition function is

$$\delta(q, k) = \begin{cases} \kappa(k) & k \in K_{\mathcal{C}} \\ \varepsilon & \text{otherwise.} \end{cases} \quad (2)$$

Figure 3.3 shows the machine. The 24 states fall into two classes: ε , and the 23 consonant states, drawn as one node c because they all behave identically. Edges are labelled *input / output*.



4 Layout Data

All layout knowledge lives in one data module, separate from the algorithm.

Key	Cons.	Key	Cons.	Key	Cons.
H	க	B	ங	[	ச
]	ஞ	O	ட	P	ண
L	த	;	ந	J	ப
K	ம	'	ய	M	ர
N	ல	V	வ	/	ழ
Y	ள	U	ற	I	ன

Table 1: Base-layer consonant keys (18). The five grantha consonants are Shift-layer keys. ஸ்ரீ is a Shift-layer special entry.

Key	Vowel	Shown on க	Sign
A	அ	க	(none)
Q	ஆ	கா	U+0BBE
S	இ	கி	U+0BBF
W	ஈ	கீ	U+0BC0
D	உ	கு	U+0BC1
E	ஊ	கூ	U+0BC2
G	எ	கெ	U+0BC6
T	ஏ	கே	U+0BC7
R	ஐ	கை	U+0BC8
C	ஒ	கொ	U+0BCA
X	ஓ	கோ	U+0BCB
Z	ஔ	கௌ	U+0BCC

Table 2: Vowel keys ν and signs σ . The pulli key is F (U+0BCD).

Because data and algorithm are separate, correcting the layout means editing these tables only.

5 The step Function

step is pure: no global state, no DOM, no React. It returns **null** for keys outside the layout.

```

1 export function step(
2   state: State,
3   code: string,
4   shiftKey: boolean = false,
5 ): { ins: string; pending: string | null } | null {
6   if (shiftKey) {
7     if (code in T.GRANTHA_CONSONANTS) {
8       const char = T.GRANTHA_CONSONANTS[code];
9       if (char === T.SHRI_CHAR) {
10         // special case: No vowel sign for ""
11         return { ins: char, pending: null };
12       }
13       return { ins: char, pending: char };
14     }

```

```

15         return null;
16     }
17
18     const isConsonant = code in T.CONSONANTS;
19     const isVowel = code in T.VOWELS;
20     const isPulli = code === T.PULLI_KEY;
21
22     if (isConsonant) {
23         const char = T.CONSONANTS[code];
24         return { ins: char, pending: char };
25     }
26
27     if (isVowel) {
28         if (state.pending !== null) {
29             if (code === "KeyA") {
30                 return { ins: "", pending: null };
31             }
32             return { ins: T.VOWEL_SIGNS[code], pending: null };
33         } else {
34             return { ins: T.VOWELS[code], pending: null };
35         }
36     }
37
38     if (isPulli) {
39         if (state.pending !== null) {
40             return { ins: T.PULLI_CHAR, pending: null };
41         } else {
42             return { ins: T.PULLI_CHAR, pending: null };
43         }
44     }
45
46     return null;
47 }

```

Keys	Output	Final state	Comment
H	க	க	consonant pending
H Q	கா	ε	vowel sign appended
H A	க	ε	inherent vowel, nothing appended
H H	கக	க	no automatic pulli
H F	க்	ε	explicit pulli
Q	ஆ	ε	independent vowel at start
Shift grantha, S	ஜி	ε	grantha takes signs
Shift ஸ்ரீ, Q	ஸ்ரீஆ	ε	no sign after ஸ்ரீ

Table 3: Example traces. H and H A show the same text but leave different states, so state cannot be recovered from the text.

A **step** call is $O(1)$ in time and space, and no typing history is needed.

6 Visualiser Design

6.1 Preview table

For each state q the visualiser needs the label of every key. The full table is built once at load by applying `step` to every pair (q, k) :

$$\text{cap}(q, k) = q \cdot \lambda(q, k).$$

This gives $|Q| \cdot |\Sigma| = 24 \times 37 = 888$ entries, each holding the inserted text, the keycap label, a no-op flag, and the key kind.

```
1 // @preview.ts
2 for (const pending of PENDING_STATES) {
3     const row: Record<string, KeyPreview> = {};
4     for (const code of ALL_CODES) {
5         const r = step({ pending }, code, false);
6         if (r) {
7             let cap = r.ins;
8             const isVowelSign =
9                 code in T.VOWELS &&
10                 pending !== null &&
11                 r.ins !== "" &&
12                 r.ins !== T.VOWELS[code];
13             const isPulli = code === T.PULLI_KEY && pending !== null;
14
15             if (pending) {
16                 if (isVowelSign) {
17                     if (pending === T.SHRI_CHAR) {
18                         // special case: No vowel sign for ""
19                         cap = T.VOWELS[code];
20                     } else {
21                         cap = pending + r.ins;
22                     }
23                 } else if (code in T.VOWELS && r.ins === "") {
24                     cap = pending;
25                 } else if (code in T.CONSONANTS) {
26                     cap = T.CONSONANTS[code];
27                 } else if (isPulli) {
28                     cap = pending + r.ins;
29                 }
30             }
31             row[code] = {
32                 ins: r.ins,
33                 cap,
34                 noop: pending !== null && r.ins === "",
35             };
36         }
37     }
38     TABLE.set(pending, row);
39 }
```

Single source of truth. The table comes from the same `step` the typing engine uses, so keycaps cannot show behaviour that differs from what happens. A unit test checks every (q, k) pair.

6.2 Architecture

- **Engine** (`tamil99.ts`, `engine.ts`, `preview.ts`): pure TypeScript, easy to unit-test.
- **Input**: one `keydown` listener reading `event.code` and the shift flag, never the OS-translated character, so it works with any active OS layout. The values are inferred from `key.Id(KeyF)`, so even the latin character F is mapped to the Tamil-99 layout.
- **Keyboard component**: renders from `previewFor(state)` and highlights held keys.

6.3 Rendering Tamil text

Each Tamil label must be shaped as one run by one font. Each keycap therefore uses a single `span`, Noto Sans Tamil listed first, no non-zero `letter-spacing`, and `lang="ta"` on the container.

7 Testing

- **Traces**: the rows of the trace table as assertions.
- **Consistency**: for all (q, k) , the preview entry equals `step`'s output.
- **Totality**: every key in Σ returns non-null in every state; keys outside Σ return `null`.
- **Encoding**: output contains only Tamil-block code points (U+0B80–U+0BFF), with no zero-width characters. For example, typing க + ஷ produces the single grapheme கஷ, without inserting a zero-width character between the components. In contrast, some other Tamil keyboard implementations may produce sequences such as கஷ with an additional zero-width character (e.g. U+200C/U+200D) in the encoded text.
- **Graphemes**: `Intl.Segmenter` keeps ெ௭ as one cluster, but splits the grantha units into two: கஷ becomes க | ஷ and ஸ்ரீ becomes ஸ | ரீ. The default Unicode rules only join virama conjuncts for some Indic scripts, not Tamil. Tests assert this behaviour so that nothing downstream relies on the segmenter to find these units; the engine tracks them explicitly.

8 Limitations and Future Work

- **Other layouts**: InScript (static) could be supported by another transducer behind the same `step/previewFor` interface.
- **Layout detection**: browsers do not expose the OS layout, so it must be inferred and user-overridable.
- **Error heat-map**: per-key error counts could be drawn on the keyboard in a typing-test extension.

9 Conclusion

Tamil99 is modelled exactly by a 24-state Mealy machine whose only memory is the pending consonant. A single pure `step` function drives both typing and the keycap labels, making the display correct by construction. The state space is small enough to pre-compute fully, and the data-driven design allows the layout to be corrected or replaced without touching the algorithm.